1. TITLE PAGE

TRANSORA
SMART CAMPUS TRANSPORT MANAGEMENT SYSTEM

Individual Project Documentation

Submitted by: Sugunesh S
Course: B.Sc. Computer Science with Artificial Intelligence
Institution: S.A. College of Arts & Science
Academic Year: 2026–2027

Submitted in partial fulfilment of the requirements for the IV Semester Summer Internship / Individual Project.

Project type: Front-end web application prototype
Technologies: HTML, CSS, JavaScript, Browser Local Storage

(Please verify and edit the student name, course, institution, semester and academic year before submission.)

2. CERTIFICATE

CERTIFICATE

This is to certify that the project titled “Transora – Smart Campus Transport Management System” is presented as an individual student project by _____ of _____ during the academic year _____.

The work described in this document represents a beginner-level prototype designed to demonstrate how campus bus information can be organised using a web interface. The project is submitted for academic review, subject to the approval and evaluation of the department.

Faculty Guide: _____
Head of Department: _____
Date: _____
Place: _____

Note: This page is a template only. Obtain the required signatures and approval from the department. Do not present it as an already-issued certificate.

3. DECLARATION & ACKNOWLEDGEMENT

DECLARATION

I declare that this project documentation describes the design and implementation of a prototype named Transora. The project demonstrates a simple web-based method for viewing and managing sample campus bus information. Any sample route names and timings shown in the prototype are illustrative and must be replaced with approved information before real use.

Student signature: _____
Date: _____

ACKNOWLEDGEMENT

I would like to thank my department and faculty members for providing the opportunity to learn through an individual project. This project helped me practise the fundamentals of HTML, CSS and JavaScript. I also learned how a simple interface can present transport information in a structured and understandable way.

The project is a prototype for learning and demonstration. It has not been connected to the college’s official transport system.

4. ABSTRACT

ABSTRACT

Transora is a Smart Campus Transport Management System prototype developed to organise basic college bus information through a web page. In a typical campus, students may need to know the bus route, departure time and current operating status. When information is shared through separate messages or informal communication, it may become difficult to find the latest details.

The proposed prototype presents bus records in a dashboard. Each record includes a bus number, route name, departure time and status. The interface displays summary counts for total buses, on-time buses and delayed buses. A transport in-charge can add a new record, update an existing bus by entering the same bus number, change its status, or remove a record.

The application uses HTML for structure, CSS for layout and visual design, and JavaScript for interactions. Browser local storage is used to retain changes on the same browser and device. The system does not require a server for this demonstration.

The main aim is to demonstrate a low-cost and easy-to-understand starting point for campus transport information management. The prototype does not provide GPS tracking, authentication, multi-device synchronisation or official notifications. These can be considered in a future version after approval and suitable backend development.

5. INTRODUCTION

1. INTRODUCTION

Transportation is an important service in many educational institutions. Students depend on college buses to travel between home and campus. Transport staff also need a clear way to organise routes, departure times and service status.

A simple digital dashboard can make this information easier to read. Transora is proposed as a lightweight web-based interface that groups the main bus details in one place. The project focuses on the information-management aspect rather than live vehicle tracking.

1.1 Background
Many small systems begin with spreadsheets, paper lists or messages. These methods may work, but details can be repeated, missed or become outdated. A web interface can display the latest data entered by the responsible staff member.

1.2 Project overview
Transora has a dashboard, a bus status list and a form for managing bus details. It is built with beginner-friendly web technologies so that the code can be understood and extended by a student.

1.3 Scope
The prototype covers sample bus records and manual status updates on one browser. It is not an official operational system and must not be used as the only source of travel information without institutional validation.

6. PROBLEM STATEMENT & NEED

2. PROBLEM STATEMENT AND NEED

Problem statement
Students and transport staff may not have one simple place to view bus route names, departure times and service status. When this information is maintained separately, it can take extra time to check or update details.

Need for the project
• To present bus information in a clear and consistent format.
• To make manual status updates simple for a transport in-charge.
• To reduce dependence on scattered sample records during a demonstration.
• To provide a foundation that can later be connected to a database.
• To practise building a practical web application using core front-end technologies.

Proposed solution
Transora displays each bus as a card with a bus number, route, departure time and status. The form lets the user add or replace a bus record. The summary area recalculates the total number of buses and status counts after changes.

Expected benefit
The prototype demonstrates how a shared layout can make transport information easier to understand. Since data is local to one browser, this version is for demonstration only. A real deployment would need a central database, user roles, data validation and reliable communication between devices.

7. OBJECTIVES & SCOPE

3. OBJECTIVES AND SCOPE

3.1 Main objective
To design and implement a simple web prototype for organising college bus routes, timings and manually updated status.

3.2 Specific objectives
1. Build a responsive page that works on common laptop and mobile browser sizes.
2. Display bus number, route, departure time and status.
3. Allow a user to add a bus record.
4. Allow a user to update a record by using an existing bus number.
5. Allow a user to remove a bus record.
6. Show summary counts that update when the list changes.
7. Preserve data in the same browser using local storage.
8. Document the design, modules, tests and limitations.

3.3 In-scope features
• Dashboard and summary counters.
• Bus list with status labels.
• Add/update and remove actions.
• Local browser persistence.
• Responsive layout.

3.4 Out-of-scope features
• GPS or live vehicle location.
• Student or staff login.
• Cloud database and multi-user access.
• SMS, email or push notifications.
• Route optimisation and payment processing.
• Integration with official college systems.

8. EXISTING AND PROPOSED SYSTEM

4. EXISTING AND PROPOSED SYSTEM

Existing approach
In a manual approach, route and timing information may be kept in paper records, spreadsheets or messages. The exact method varies by institution. If updates are not made consistently, users may see old information.

Limitations that may occur
• Information can be distributed across multiple places.
• A person may need to ask staff for a status update.
• A list may not clearly show which buses are delayed.
• Repeated manual counting can take time.
• Changes may not be visible to everyone immediately.

Proposed approach
Transora uses a single dashboard to present sample transport details. The status is represented with a text label and distinct visual style. A form provides a consistent way to add and update records.

Comparison
Manual list: simple to start, but presentation and update consistency depend on the person maintaining it.
Transora prototype: clear web display and automatic counters, but storage is local to one browser.
Future centralised system: can share validated information across devices, but requires backend services, authentication, hosting and maintenance.

The project does not claim that the prototype solves every transport problem. It demonstrates a first step toward structured information management.

9. REQUIREMENTS & TECHNOLOGIES

5. REQUIREMENTS AND TECHNOLOGIES

5.1 Hardware requirements
• Laptop or desktop computer.
• Keyboard and pointing device.
• Sufficient storage for project files.
• Internet connection is optional for running the local HTML file; it may be needed to install development tools.

5.2 Software requirements
• Windows, macOS or Linux operating system.
• Modern web browser such as Chrome, Edge or Firefox.
• Visual Studio Code or another text editor.
• Optional Live Server extension for convenient local testing.

5.3 Technologies used
HTML: defines the page structure, headings, forms, buttons and content sections.
CSS: controls colours, spacing, cards, layout, responsiveness and status styles.
JavaScript: handles form submission, record updates, removal, counter calculation and rendering.
Browser local storage: stores the current demo records in the browser on the same device.

5.4 Why these technologies?
They are widely used for basic web development and are suitable for a small front-end prototype. No database server or external library is required for the current version.

5.5 Data fields
Bus number: identifier used to recognise a record.
Route name: text describing the sample route.
Departure time: text representing the scheduled departure.
Status: On-Time, Delayed or Not-Running.

10. SYSTEM DESIGN & WORKFLOW

6. SYSTEM DESIGN AND WORKFLOW

High-level workflow
1. The user opens index.html in a browser.
2. JavaScript attempts to read saved bus records from local storage.
3. If no saved records are found, the sample records are displayed.
4. The dashboard renders each bus and calculates summary counts.
5. The in-charge enters bus details and selects a status.
6. On saving, JavaScript validates required fields and checks whether the bus number already exists.
7. If the bus number exists, the record is updated; otherwise a new record is added.
8. The updated list is saved to local storage and displayed again.
9. The user can remove a record or reset the sample data.

Logical modules
• Dashboard module: shows total, on-time and delayed counts.
• Bus display module: renders bus cards and status labels.
• Bus management module: handles adding and updating.
• Removal module: removes the selected record.
• Storage module: saves and retrieves records from local storage.

Design principle
The page keeps actions visible and uses plain labels. The design aims to be understandable to a first-time user. For real deployment, role-based access and server-side validation should be added.

11. IMPLEMENTATION DETAILS

7. IMPLEMENTATION DETAILS

7.1 Page structure
The HTML page is divided into a header, introduction section, dashboard statistics, bus list, management form and footer. Navigation links take the user to the main sections.

7.2 Styling
CSS variables define the dark background, panel colours, cyan highlight and orange accent. Cards use rounded corners and spacing to separate content. A media query changes the layout for narrower screens.

7.3 Data handling
The JavaScript array named buses holds the current records. The render function clears the visible list, creates a card for each bus, and updates the summary counts. The form submit handler reads the entered values and saves the record.

7.4 Updating a record
The bus number acts as the simple matching key. If the entered number matches an existing record, its route, time and status are replaced. Otherwise, a new record is appended.

7.5 Persistence
The save function serialises the array as JSON and writes it to local storage. When the page opens again in the same browser, the saved records can be restored.

7.6 Limitations of this implementation
Local storage is not a shared database. Data is not synchronised to other devices, and users are not authenticated. The prototype is not suitable for official transport operations without further development and testing.

12. USER GUIDE & TESTING

8. USER GUIDE AND TESTING

User guide
1. Open the Transora folder in Visual Studio Code.
2. Open index.html in a browser, or use Live Server if installed.
3. Review the sample buses shown in the Bus Status section.
4. In Manage Bus Details, enter a bus number, route and departure time.
5. Select On-Time, Delayed or Not-Running.
6. Click Save Bus Details.
7. Confirm that the bus card and summary counts change.
8. Enter an existing bus number to update that record.
9. Click Remove to delete a record.
10. Click Reset Demo to restore the three sample records.

Test cases
Test 1: Open the page. Expected: dashboard and sample buses appear.
Test 2: Add a new bus with all fields. Expected: new card appears and total count increases.
Test 3: Reuse an existing bus number with different details. Expected: existing record updates, not duplicates.
Test 4: Select Delayed. Expected: status label displays Delayed and delayed count updates.
Test 5: Remove a bus. Expected: card disappears and total count decreases.
Test 6: Refresh the browser. Expected: saved changes remain in the same browser.
Test 7: Submit with a required field empty. Expected: browser required-field validation prevents submission.
Test 8: Reset demo. Expected: three example records return.

Record the actual result of each test after running the project. Add screenshots from your own test run to the final report.

13. RESULTS & SCREENSHOTS

9. RESULTS AND SCREENSHOTS

Expected result
The Transora prototype should display a dashboard with three sample buses and summary counts. The user should be able to add a record, update an existing record, change its status, remove a record and reset the demo data.

Screenshots to capture
Figure 1: Transora home page and introduction.
Figure 2: Transport overview showing the summary counters.
Figure 3: Bus status cards showing sample route and time information.
Figure 4: Management form filled with a new sample bus.
Figure 5: Updated bus list after saving.
Figure 6: Status counter after changing a bus to Delayed.
Figure 7: Browser view after refreshing to confirm local persistence.

Screenshot instructions
Run index.html in your browser. Capture screenshots using Windows Snipping Tool (Win + Shift + S) or another screenshot utility. Insert only screenshots from your actual running project. Do not use unrelated images or claim untested results.

Result evaluation
Mark each test as Pass or Fail after you run it. If a test fails, note the issue and correct it before final submission. This report describes expected behaviour; the final result section should be updated to match the actual test run.

14. LIMITATIONS & FUTURE ENHANCEMENTS

10. LIMITATIONS AND FUTURE ENHANCEMENTS

Current limitations
• The application stores information in one browser only.
• It does not provide live GPS tracking.
• It does not support staff accounts or access permissions.
• It does not send alerts to students.
• Sample routes and timings are not verified official transport data.
• Data may be lost if browser storage is cleared.
• It has not been security-reviewed or deployed for production use.

Future enhancements
1. Add a backend using a suitable server framework.
2. Use a central database to share updates across devices.
3. Create secure staff login and role-based permissions.
4. Add a student view that is read-only.
5. Provide route-wise and stop-wise details.
6. Add notifications after the college approves the workflow.
7. Consider GPS integration only if approved, affordable and privacy-conscious.
8. Add validation and audit history to record who changed a bus status.
9. Host the application securely and provide backups.
10. Conduct usability testing with students and transport staff.

A future version should be planned with guidance from the college. Any real student, staff or transport data should be handled with permission and suitable privacy safeguards.

15. CONCLUSION & REFERENCES

11. CONCLUSION

Transora demonstrates a simple approach to displaying and managing campus bus information through a web interface. The prototype brings bus numbers, routes, departure times and service status into one page. It also demonstrates add, update, remove, reset and local storage features.

The project provided practical experience with HTML, CSS and JavaScript. It shows how a small web application can be structured into clear modules and tested using simple test cases. The prototype is intentionally limited: it does not offer GPS tracking, a shared database, user authentication or live cross-device updates.

The next stage would be to consult the department, confirm the actual requirements and build a secure backend if a shared system is needed. The project should be evaluated by the faculty before being treated as an official transport solution.

REFERENCES

1. MDN Web Docs, HTML: https://developer.mozilla.org/en-US/docs/Web/HTML
2. MDN Web Docs, CSS: https://developer.mozilla.org/en-US/docs/Web/CSS
3. MDN Web Docs, JavaScript: https://developer.mozilla.org/en-US/docs/Web/JavaScript
4. MDN Web Docs, Window.localStorage: https://developer.mozilla.org/en-US/docs/Web/API/Window/localStorage
5. Visual Studio Code documentation: https://code.visualstudio.com/docs

Appendix: The project source file is index.html. The README.txt file contains setup and testing instructions.

Before submission, edit all personal/college details, run the project, add actual screenshots, record real test results, and confirm the department's required format.